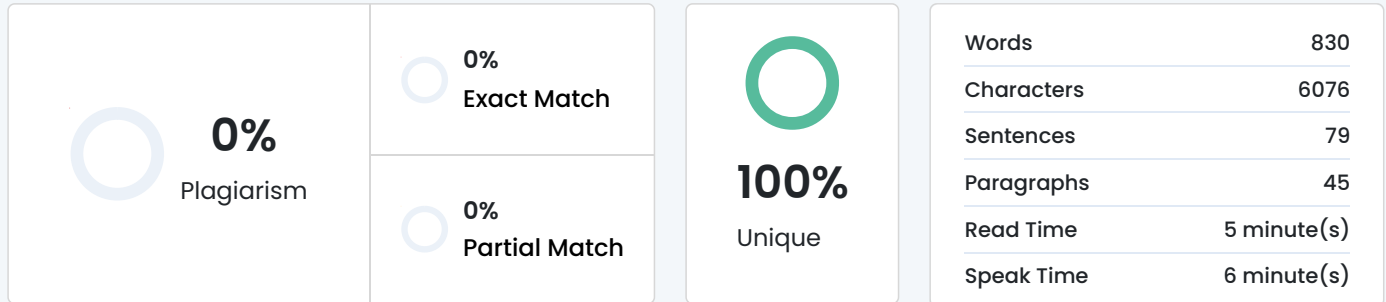


Plagiarism Scan Report



Content Checked For Plagiarism

Chapter 1 Overview of the Company

1.1 About the Company

Meditab Software (I) Pvt. Ltd. is a leading provider of software in the healthcare sector, creating innovative products such as Intelligent Medical Software (IMS). Founded in 1998, Meditab has built a legacy of bringing together comprehensive software implementations for hospitals, dispensaries, pharmacies, and specialty clinics. The organizational culture nurtures research and development, aiming to optimize the healthcare technology landscape. Operations are distributed internationally, focusing on quality assurance, HIPAA compliance, and agile software development life cycles.

1.2 Key Offerings

The company offers an extensive suite of products beyond just EHR (Electronic Health Record). Their flagship product, IMS, provides deep integrations into practice management and medical billing. Additionally, they provide intelligent scheduling algorithms, automated prescription fulfillment software, and a robust telemedicine connectivity suite designed to empower patients. Over the years, Meditab has transitioned to cloud-based solutions heavily reliant on large-scale databases and microservice-driven backends.

1.3 Company Growth & Recognition

Meditab has experienced continuous growth by adapting to modern technologies and meeting regulatory compliances in the US healthcare market, such as Meaningful Use certifications. The company invests heavily in automating internal QA processes to handle the large testing matrices effectively, minimizing regression cycles and time-to-market. Quality assurance is treated as a first-class citizen.

1.4 Scope of Work

The intern's scope of work involves developing internal tools for Quality Assurance, specifically streamlining automation testing through a centralized dashboard known as AXES. By automating the execution triggers across remote server nodes, the intern minimizes the manual effort typically required to perform complex build pipelines. The scope entails full-stack development covering a Node.js framework and React.js interfaces.

Chapter 2 Overview of Internship

2.1 Internship Summary & Project Details

The internship focused on the development of IMSAxes, a robust automation execution platform integrating front-end and back-end environments to facilitate seamless automation test triggering, tracking, and reporting. The internship exposed the intern to real-world software engineering dynamics, sprint planning, and the rigorous discipline of maintaining enterprise-grade architectural standards. It covered the deployment of Node instances, securing APIs via stateless tokens, and building responsive SPAs (Single Page Applications) utilizing the React ecosystem.

2.2 Project Overview

IMSAxes is a comprehensive system comprising a Node.js backend (AXES) and a React frontend (Axes_redux). It handles execution setups, schedules test runs via Maven strings, collects live logs via Socket.io bidirectional connections, and provides real-time tracking of executing suites without refreshing the browser.

2.3 Purpose of Project

The purpose is to minimize manual intervention during automation execution. Historically, QA engineers had to manually log in using Remote Desktop to virtual machines, update the Git repository, configure the execution XML files, and run Maven commands. This project fully automates this pipeline. It provides the QA team with an isolated environment to execute automated scripts uniformly and track issues transparently.

2.4 Objective

To build a scalable and interactive platform for automated sanity and regression testing. The key objectives include: 1) Reducing manual testing configuration effort by 80%. 2) Increasing test coverage visibility. 3) Providing a fault-tolerant HTTP API capable of handling concurrent testing sessions.

2.5 Technology & Tools

The project leveraged a modern web stack:

- Frontend: React.js (Component-based UI), Redux (Global State Management).
- Backend: Node.js, Express framework for routing.
- Database: MySQL managed via Sequelize ORM.
- Execution Environment: Windows Server, PowerShell scripts for OS-level control.
- Automation tools: Maven, Socket.IO for web sockets, Websocketify for VNC proxying.

Chapter 3 Project Structure and Management

3.1 Project Planning

The project was planned with distinct milestones adhering to an Agile methodology. Initial scoping involved gathering requirements from QA leads. The back-end restructuring phase decoupled the orchestration engine from the data access layer. Subsequent phases implemented the Websocket servers, followed by dynamic state mapping in the React frontend.

3.2 Function Points and Complexity

The system was divided into various modules to calculate effort and lines of code (LOC). The complexity was distributed primarily between the UI rendering and the backend task orchestration mechanisms.

3.3 Lines of Code (LOC) Estimates

The system was divided into various modules to calculate effort and lines of code (LOC). The complexity was distributed primarily between the UI rendering and the backend task orchestration mechanisms.

Chapter 4 Project And System Requirement Study

4.1 User Characteristics

The application serves QA Engineers, Developers, and System Administrators. Users are expected to have fundamental knowledge of testing terminologies like Regression, Sanity, Test Suites, and pass/fail diagnostics.

4.2 Hardware And Software Requirements

Hardware Specifications:

- Server Node: Intel Xeon E5 or equivalent, 16GB RAM, 100GB SSD Storage.
- Client Node: Intel Core i5, 8GB RAM, Modern internet connection.

Software Specifications:

- Node.js v14+, NPM/Yarn.

- MySQL 8.0 instance running locally or on a dedicated DB cluster.
- Target IIS or Express Server hosting.

4.3 Assumptions and Dependencies

It is assumed that the remote IMS target servers (e.g., `imsautomate02`) are continuously active and connected to the corporate intranet. Correct execution credentials must be preemptively registered in the database. Network firewalls must allow inbound/outbound traffic on the configured Socket.IO port.

Matched Source

Congratulations!

No Mistake Found

Check By:  Dupli Checker